

Personal Running Dashboard with AI-Powered Run Analysis

Project Specification — Architecture, Technology Stack & Cost Plan

Prepared: June 2026 | Status: Planning complete, ready for build

1. Project Overview

The goal of this project is a personal running dashboard that automatically visualizes training data pulled from Garmin Connect, with an AI-generated analysis attached to every run as soon as it is synced — with no manual chat interaction required after each run.

The project is split into two build phases:

- **Phase 1 — Data & Visuals:** pull detailed Garmin activity and health data, store it, and present it on a personal dashboard.
- **Phase 2 — AI Layer:** automatically generate a coaching-style analysis for each new run and surface it inside the dashboard.

2. Problem Statement & Key Decisions

Two data-source paths were evaluated before settling on the current approach:

- **Garmin's official Activity API** (Garmin Connect Developer Program) is free but gated behind approval as a *business* developer. It is not a realistic path for a personal hobby project.
- **Strava** (where Garmin activities already auto-sync) aggregates interval and tempo workouts without preserving the structured, per-repetition detail needed for a meaningful analysis — it visually lumps the workout together.

Decision: use **GarminDB**, an open-source tool that authenticates directly against Garmin Connect and downloads the full **.FIT** files for every activity — preserving the same lap, split, and second-by-second detail visible in the Garmin Connect app itself.

3. Data Source: GarminDB

GarminDB (github.com/tcgoetz/GarminDB) was reviewed directly. Key findings:

- Downloads full **.FIT** activity files, plus daily monitoring (heart rate, steps, stress, intensity minutes), sleep, weight, and resting heart rate.
- Parses each activity into separate, queryable tables: **ActivityLaps** (each repetition/lap), **ActivityRecords** (point-by-point pace/HR/cadence), and **ActivitySplits**.
- Stores everything locally in **SQLite** databases — structured and ready to query, no separate database server needed.

- Authenticates via the unofficial **python-garminconnect** library, with MFA support and cached session tokens (no repeated login prompts).
- Ships a CLI flag, `--latest`, that performs an incremental sync — only pulling what is new since the last run. This is the hook the AI layer (Phase 2) attaches to.

Caveat: *this relies on an unofficial, reverse-engineered access method, not an Garmin-endorsed integration. It is widely used in the self-hosted fitness-data community and has been actively maintained, but it can break if Garmin changes its login flow, and it sits in a gray area relative to Garmin's terms of service.*

4. System Architecture

End-to-end data and analysis flow:

Step	What happens
1	Garmin device syncs the run to Garmin Connect (existing behavior, unchanged).
2	A scheduled job (cron) on the always-on VM runs GarminDB with <code>--latest</code> .
3	GarminDB authenticates, downloads new .FIT files, and updates the local SQLite databases.
4	FastAPI backend reads SQLite and exposes the data as JSON endpoints.
5	React dashboard renders charts and per-run views from those endpoints.
6 (Phase 2)	The same scheduled job detects the newest activity, formats its laps/splits/HR into a prompt, and calls the Claude API.
7 (Phase 2)	The generated analysis is stored (new table or JSON) and displayed next to the matching run in the dashboard.

5. Technology Stack

Layer	Technology
Data collection	GarminDB (Python) + python-garminconnect
Database	SQLite (local file, managed by GarminDB)
Scheduling	cron, on the hosting VM
Backend / API	FastAPI (Python) — reads SQLite, serves JSON
Frontend	React + Vite, charts via Recharts
Hosting (24/7)	Oracle Cloud Free Tier — Always Free ARM VM

Layer	Technology
Private access	Tailscale — private network between phone and PC, no public domain needed
AI provider (Phase 2)	Anthropic Claude API — Claude Sonnet 4.6
Build tool	Claude Code

6. Hosting & Access Plan

Hosting: Oracle Cloud Free Tier

Chosen to keep recurring infrastructure cost at \$0. The Always Free tier includes an ARM-based VM with enough resources to comfortably run GarminDB, the cron job, the SQLite database, the FastAPI backend, and the dashboard.

Known friction: provisioning the free ARM instance can fail with an “out of host capacity” error in busy regions, purely due to demand. If this happens repeatedly, retry at a different time of day or a different availability domain.

Fallback: a small paid VPS (e.g. Hetzner, ~\$4–5/month) removes this friction entirely.

Access: Tailscale

The dashboard is for personal use only, from a phone and a PC. Tailscale creates a private network between those devices and the VM at no cost, with no public exposure, no domain purchase, and no manual HTTPS/certificate setup required.

7. Phase 2: AI Analysis Layer

After each GarminDB sync, a script checks for a new activity, builds a structured prompt from its laps, splits, pace, and heart-rate data, and sends it to the Claude API. The returned analysis is stored and shown alongside the run in the dashboard — no chat session required.

Model recommendation: Claude Sonnet 4.6. At this usage volume (see cost analysis below), the price difference versus a cheaper model is negligible, so it makes sense to default to the higher-quality analysis.

8. Cost Analysis

Phase 1 — Data & Visuals

Item	Notes	Cost
GarminDB	Open source	\$0
Hosting (Oracle Cloud Free Tier)	Always Free ARM VM	\$0 / month
Hosting fallback (Hetzner, if needed)	Small VPS	~\$4–5 / month
Private access (Tailscale)	Free personal tier	\$0
Domain / SSL	Not needed (private access via Tailscale)	\$0

Phase 2 — AI Analysis

Anthropic's published API rates (verified June 2026): Claude Sonnet 4.6 costs \$3 per million input tokens and \$15 per million output tokens.

Estimated usage at ~20 analyzed runs per month (4–5 runs/week):

Component	Basis	Cost
Input tokens	~2,000 tokens/run × 20 runs = 40,000 tokens	\$0.12
Output tokens	~600 tokens/run × 20 runs = 12,000 tokens	\$0.18
Estimated monthly total	Sonnet 4.6, standard rates, no caching/batch	~\$0.30

Even with a generous margin on prompt size, this stays under roughly \$1/month at the stated usage volume. There is no permanent free API tier, but new Anthropic accounts receive starter credits that comfortably cover the first several months at this volume. Verify current rates at claude.com/pricing before final budgeting, since pricing can change.

Development Cost (Claude Code)

Building the project happens through Claude Code. If an existing Claude Pro or Max subscription is already in use, Claude Code usage is included within that plan's limits, and a project of this scope is unlikely to exceed them. No separate budget line is assumed for the build phase itself.

Total Recurring Cost Summary

Item	Monthly Cost
Phase 1 (data + dashboard)	\$0 / month
Phase 2 (AI analysis)	~\$0.30–1.00 / month
Development (Claude Code)	\$0 extra, if Pro/Max already active
Estimated total	Under \$1–2 / month

9. Suggested Build Order

- Provision the Oracle Cloud VM and connect it via Tailscale.
- Install and run GarminDB; confirm lap/split-level detail matches what is seen in the Garmin Connect app.
- Build the FastAPI layer that reads SQLite and exposes the data.
- Build the React dashboard and core visualizations.
- Set up the cron job for incremental (--latest) syncing.
- Build the AI analysis script and connect it to the Claude API.
- Wire the generated analysis into the dashboard, next to each run.

This document reflects the architecture, technology choices, and cost estimates discussed and verified during planning. It is intended as the starting reference for the Claude Code build session.